Databricks Data Engineering — Hands-On Practice

Sample data + a notebook that generates Parquet and walks the full flow

explore → clean → star schema → Delta → MERGE → windows → time travel/OPTIMIZE

Practice real Databricks data engineering on a messy e-commerce dataset. Includes **sample CSV data**, a **Databricks notebook** that generates **Parquet** and walks you through the full flow, and this step-by-step guide.

What's here

```
databricks-practice/
├── databricks_de_practice.py   # the notebook — import into Databricks and run cell by cell
├── sample_data/                # CSV inputs (customers, products, orders) — optional
│   ├── customers.csv           # 200 customers
│   ├── products.csv            # 50 products
│   └── orders.csv              # 10,150 messy order rows (dupes, nulls, cancelled, bad refs)
└── README.md
```

About the Parquet files

Parquet is a **compressed, binary** format, so you create it *inside* Databricks (Spark writes it in one line). The notebook's **Step 1** generates the sample data and **writes Parquet to DBFS** — that's your practice `.parquet` files. (If you'd rather start from files, upload the CSVs and read those instead.) *Optional:* if you have Python with `pandas` / `pyarrow` locally, you can turn the CSVs into Parquet yourself with the snippet at the bottom.

Step-by-step approach (what the notebook does & why)

0. Setup — pick a DBFS working path (`/tmp/de_practice`) with Bronze/Silver/Gold zones.

1. Generate + write Parquet (Bronze/raw) — build 3 datasets as Spark DataFrames and `df.write.parquet(...)`. The orders are deliberately messy so there's real work to do.

```
ord_df.write.mode("overwrite").parquet(f"{RAW}/orders")   # creates your Parquet files
```

2. Read & explore — the first thing you always do:

```
orders = spark.read.parquet(f"{RAW}/orders")
orders.printSchema(); orders.count(); display(orders.limit(10))
display(orders.groupBy("status").count())     # spot the mess
```

3. Bronze → Silver (clean) — the heart of ETL. Keep **completed** orders with **positive amount**, **dedupe** by `order_id` (latest via a window), and enforce **referential integrity** by joining to valid customers/products:

```
w = Window.partitionBy("order_id").orderBy(F.col("order_ts").desc())
silver = (orders.withColumn("amount", F.col("amount").cast("double"))
    .filter((F.col("status")=="completed") & (F.col("amount")>0))
    .withColumn("rn", F.row_number().over(w)).filter("rn=1").drop("rn")
    .join(F.broadcast(customers.select("customer_id")), "customer_id")
    .join(F.broadcast(products.select("product_id")), "product_id")
    .withColumn("order_date", F.to_date("order_ts")))
silver.write.mode("overwrite").partitionBy("order_date").parquet(f"{SILVER}/orders")
```

4. Gold — dimensional model (star schema) — `fact_orders` + `dim_customer/product/date`, written as **Delta** tables.

5. Delta tables + SQL — register the Gold tables and query them with SQL (revenue by region). Delta = Parquet + **ACID, MERGE, time travel, OPTIMIZE**.

6. MERGE (upsert) — the incremental/CDC pattern: update existing rows + insert new ones in one atomic operation.

```
(fact.alias("t").merge(updates.alias("s"), "t.order_id = s.order_id")
   .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())
```

7. Window functions — top-N orders per customer, running total of spend.

8. Analytics — top products by revenue (broadcast join), daily revenue.

9. Delta features — **time travel** (`versionAsOf`), **OPTIMIZE** (compact small files), **Z-ORDER** (data skipping), and **VACUUM** (cleanup).

10. Recap — you've practiced the medallion architecture, Delta, MERGE, windows, and tuning — exactly what Databricks DE interviews and take-homes test.

How to run it

Option A — Databricks (recommended, free):

1. Sign up for **Databricks Community Edition** (free) → create a cluster.
2. **Workspace** → **Import** → **File** → upload `databricks_de_practice.py` (it imports as a notebook).
3. Attach the cluster and **Run all** (or run cell by cell to learn).

- You don't need the CSVs — Step 1 generates the data and Parquet for you.

Option B — use the CSVs as input: Upload `sample_data/*.csv` via **Data** → **Add Data**, then read them:

```
orders = spark.read.option("header", True).option("inferSchema",
True).csv("/FileStore/tables/orders.csv")
```

…and continue from Step 3.

Option C — local PySpark: `pip install pyspark`, then run the same code in a local notebook (Delta needs the `delta-spark` package + config; Databricks has it built in).

Practice challenges (level up)

1. Add an **SCD Type 2** `dim_customer` — MERGE that closes the old row (end-date + flag) and inserts a new version when `region` changes.
2. Compute **month-over-month revenue growth** with `LAG`.
3. **Sessionize** clicks (new session after 30 min inactivity) using `LAG` + a running sum.
4. Try **Auto Loader** (`spark.readStream.format("cloudFiles")`) to ingest files incrementally.
5. Build a **Delta Live Tables** pipeline with `@dlt.expect` quality rules.

Optional: make Parquet from the CSVs locally (if you have pandas/pyarrow)

```
import pandas as pd
for name in ["customers","products","orders"]:
    pd.read_csv(f"sample_data/{name}.csv").to_parquet(f"sample_data/{name}.parquet",
index=False)
```

Then upload the `.parquet` files to Databricks and `spark.read.parquet(...)`.